

C30 — Five End-to-End Product Journeys

1. What interviewers test

"Walk me through a product you took from an idea to a launch." "How would you build this from nothing, and how would you know when to stop?" Interviewers rarely test a single skill in isolation; they test whether the skills fit together: whether discovery changes the spec, the spec defines what to measure, the first version tests the riskiest assumption, the kill criteria are set before the results, and the retrospective changes the next decision. This chapter shows five complete journeys, one product at a time, so you can see the earlier chapters working together. The products and every number are invented; their purpose is to demonstrate the connections and the reasoning, which you can then apply to your own work and to open-ended product questions.

Where it shows up. **Inferred**: end-to-end product and execution questions at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and others, and the behavioral prompt "tell me about a product you shipped" **[unverified: candidate-reported]**. On the job it is the shape of every project.

What is scored. Whether your account of a product has a through-line **Inferred**: a problem with evidence, a riskiest assumption, a cheap test with a threshold, a spec that states non-goals and acceptance criteria, a metric tree with guardrails, a decision to continue or stop, and a lesson that changed a habit.

Six observable behaviors:

1. You connect each stage to the evidence from the stage before.
2. You name the riskiest assumption and test it with the cheapest experiment that can fail.
3. You write a spec with a problem statement, goals, non-goals and acceptance criteria.
4. You build a metric tree with a primary measure, drivers and guardrails.
5. You set kill criteria before the results arrive.
6. You close with a retrospective that names a pattern and a changed practice.

The failure this prevents. Six patterns end end-to-end answers: a list of activities with no decisions, a spec written after the build, metrics chosen after the results, no kill criteria, a launch described as the finish line, and a retrospective that is a list of virtues.

2. The concept from first principles

C30.1 How to use the journeys

These are illustrative, not case studies. Every product, team, interview, test result and figure in section 5 is invented (invented), built to teach how the pieces connect. They are not records of any real company or product, and the numbers are not benchmarks. A real, public lifecycle model is reviewed in CS-C30-02.

Read each journey three times.

1. *For the structure:* cover the results and write what you would do at each stage; compare.
2. *For the reasoning:* at each decision, find the evidence that drove it and ask what else could have been decided.
3. *For the numbers:* recompute the figures and note which assumption each rests on.

Do not memorize them. An interviewer who asks about a project of yours will probe the details (C31.7); a memorized journey collapses at the second question. Use the journeys to build the habits and the vocabulary, then tell your own story from your own work (CS-C30-01).

Use them as templates for open-ended questions. When asked to "build a product for X," the journey template (FW-C30-01) gives a skeleton for a 30-minute answer: the user and problem, the evidence you would gather, the riskiest assumption and its test, the first version, the metric tree and the decision rule.

Counterexample. Asked to walk through a product they shipped, a candidate says: "We did user research, then design, then engineering built it over three months, and we launched in June." The interviewer asks what the riskiest assumption was and what would have made the team stop. The candidate has listed activities and cannot name a decision, a threshold or a result. A better start: "The riskiest assumption was that parents would pay, so before building we ran a manual version for 40 families with a threshold of 10% paying and a stop below 5%; it reached 15%, so we built."

When not to use the full template. A small, reversible change needs a paragraph and a staged rollout, not seven parts and a kill-criteria card (C17). Do not present the template as how the work actually happened if it was messier: say where you were wrong, because a tidy story invites doubt (C31.7). And do not force a journey structure onto a question that only asks for one stage; answer the stage asked.

C30.2 Five journeys at a glance

#	Journey	Distinct challenge	Chapters exercised	Setting
1	A consumer learning companion for exam preparation, with an AI tutor	Trust in AI answers; low-data use; a paying parent and a using student	C06, C20, C24, C26	India (low-end phones, regional languages)
2	An enterprise operations tool for warehouse exceptions	A long sales cycle; security review; integration; roadmap commitments	C16, C17, C27	Global
3	A two-sided marketplace with a cold start: small shops and wholesalers	Supply and demand together; credit; manual start	C19, C26, C27	India (neighborhood retail)
4	An internal AI evaluation platform	Customers are other teams; adoption without a mandate; measuring a judge	C21, C25, C27	Global
5	An autonomous agent for knowledge work (purchase requests)	Reliability of a chain; approvals; injection; earned autonomy	C24, C25, C32	Global

At most two journeys are India-specific (1 and 3); the others are set in a general global context.

C30.3 The per-journey template

Each journey in section 5 follows the same seven parts (FW-C30-01), so you can compare them:

1. **Context and users.** Who, what job, what alternative.
2. **Discovery.** The evidence gathered, the riskiest assumption, the cheapest test.
3. **Spec excerpt.** Problem statement, goals, non-goals, key requirements, acceptance criteria (C16).
4. **Metrics tree.** Primary measure, drivers, guardrails (C05, C27.10).
5. **MVP, kill criteria and results.** The smallest test, the thresholds set in advance, the readout (C26).
6. **Tradeoffs.** The decisions with their alternatives and costs.
7. **Retrospective.** What went wrong, the anti-pattern (C29), the practice changed.

Where each part is taught: discovery in C06 and C26; the spec in C16; metrics in C05 and C27; kill criteria in C26; rollout in C17; AI specifics in C24 and C25; domain concepts in C27; communication in C32; failure patterns in C29.

C30.4 Closing: what the five journeys teach

Reading across the journeys shows recurring structure.

- **The riskiest assumption was different each time, and it was never the build.** Willingness to pay and trust (1), the buying process and integration (2), supply behavior (3), adoption (4), reliability and trust in approvals (5). The cheapest test addressed that assumption first.
- **A manual or narrow first version beat a built one.** A concierge test in 1 and 3, three pilot sites in 2, five teams in 4, a draft-only mode in 5.
- **Kill criteria were set before the data and the results were mixed.** In several journeys the result passed one criterion and not another, and the decision came from the pre-set rule plus judgment about the segment.
- **Non-feature requirements shaped the plan.** Security review (2), parental consent and data limits (1), credit and payments rules (3), permissions and cost (4), injection and audit (5).
- **Each retrospective named an anti-pattern from C29** and a practice change with a check: roadmap as a promise (2), a vanity count (4), the demo as evidence (5), a test sample that was too easy (1), a supply-side target (3).

What the journeys do not show. They are tidy: real projects have false starts, politics and changes of mind. The value is in the connections between stages, which you should be able to reproduce with your own, messier, evidence.

3. Frameworks

Three frameworks, set out below. The problem-definition template (C02.7), the PRD template (C16), the metric card (FW-C05-01), the riskiest-assumption map and test card (FW-C26-04, FW-C26-05), the kill-persist-pivot table (FW-C26-06), the autonomy ladder (FW-C24-05), the trust progression (FW-C25-05), the anti-pattern scan (FW-C29-01) and the failure-story structure (FW-C29-03) are reused, not repeated.

FW-C30-01 · The journey template · Original

- **Inputs:** a product idea or a product you shipped.
- **Outputs:** a one-page account in seven parts: context and users; discovery (evidence, riskiest assumption, test); spec excerpt (problem, goals, non-goals, requirements, acceptance criteria); metrics tree with guardrails; MVP with kill criteria and results; tradeoffs; retrospective with a pattern and a practice.
- **Assumptions:** each part is justified by the evidence of the part before.
- **Tradeoffs:** a template makes the through-line visible and can make a messy project look tidier than it was; say where it was not.
- **Failure modes:** a spec with no connection to discovery; metrics chosen after results; no retrospective.
- **Worked example:** the five journeys in section 5.

Part	Content
Context and users	
Discovery: evidence, riskiest assumption, test	
Spec excerpt	
Metrics tree and guardrails	
MVP, kill criteria, results	
Tradeoffs	
Retrospective: pattern, practice	

FW-C30-02 · Phase gates with exit criteria · Original

- **Inputs:** a product moving from idea to live.
- **Outputs:** four phases (discovery, alpha, beta, live), each with its purpose, the evidence required to exit, the decision (proceed, persist, pivot or stop), and the reviewers; adapted from the phase-and-assessment model that public-sector digital teams use (CS-C30-02).
- **Assumptions:** exit criteria are written before the phase starts.
- **Tradeoffs:** gates add process and prevent building ahead of evidence.
- **Failure modes:** gates with no criteria; one gate skipped under pressure; a live phase with no continued review.
- **Worked example:** CS-C30-02 and journeys 2 and 5.

Phase	Purpose	Exit evidence	Decision
Discovery	Understand users and the problem	Evidence of the problem; the riskiest assumptions; the plan for alpha	Proceed, stop
Alpha	Test different solutions to the riskiest assumptions	A tested solution with results against thresholds	Proceed, pivot, stop
Beta	Build and refine the best option with real users (private, then public)	Working end-to-end service; metrics and guardrails; operational readiness	Release wider, hold
Live	Run and keep improving	Continued review of outcomes; a plan for the next cycle	Improve, narrow, retire

FW-C30-03 · The kill-criteria card · Original

- **Inputs:** a test or a phase.
- **Outputs:** for each criterion: the measure, the threshold to proceed, the threshold to stop, the sample size, the date, and the decision for the middle zone (persist with a better test, narrow, or pivot); written before the test and dated.
- **Assumptions:** you agree with stakeholders who decides on the result.
- **Tradeoffs:** written criteria can be wrong; revise them only by an agreed process, not after results.

- **Failure modes:** thresholds set after results; no middle zone; a measure that can be gamed.
- **Worked example:** journeys 1 to 5 and EX-C30-01.

Measure	Proceed if	Stop if	Sample	Date	Middle zone
---------	------------	---------	--------	------	-------------

4. Answer architecture

The "take me through a product from idea to launch" answer, for 30 minutes

Step	Minutes	What to produce
Context, user and the evidence	5	The problem, who has it, what you learned and how
The riskiest assumption and the test	6	Why it was the riskiest; the cheapest test; the threshold set in advance
The spec and what you left out	5	Problem statement, goals, non-goals, acceptance criteria
Metrics and guardrails	4	The tree; the primary measure; a guardrail
The first version and the result	5	What you built, what happened, the decision by the criteria
Tradeoffs and retrospective	5	A decision with its alternative; a pattern and a changed practice

Pacing rules. If the interviewer probes a stage, go deep there and use the template to return. If you do not have a real project, use a product you studied or a case you reconstruct, and say so.

How to say it. "The riskiest assumption wasn't the build, it was whether parents would pay, so we tested that first with a manual version for 40 students, with a threshold of 10% paying and a stop below 5%."

How to close. "The result passed two of three criteria and I decided by the rule we'd set; the lesson was that my first sample was too easy, so I now review a stratified sample."

Type: walk me through a product you shipped

The template with your real numbers; the through-line; one thing that went wrong and the practice you changed.

Type: build a product for X from scratch

User and problem, evidence you would gather, riskiest assumption and test, first version, metric tree, decision rule.

Type: what would you do in the first 90 days of this product

Phase gates: what discovery must answer, what alpha tests, what beta needs to be ready, with criteria.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"How did you know that?"	Evidence for a stage	Name the interviews, data or test and the number
"What was the riskiest part?"	Assumption thinking	The assumption, why it was riskiest, how you tested it
"What would have made you stop?"	Kill criteria	The thresholds you set before the result
"What did you leave out?"	Non-goals	A real non-goal and why
"What went wrong?"	Honesty and learning	A pattern, a root cause, a changed practice
"How is this different from your last project?"	Transfer	What stayed the same (the template) and what changed (the riskiest assumption)

5. Worked examples

Five **Illustrative** journeys. Every product, person, test and figure is invented (invented), and the arithmetic was computed. Names such as StudyBuddy are made up and do not refer to any real product.

Worked example 1 — Journey 1: a learning companion for exam preparation (Illustrative)

India-specific; exercises C06, C20, C24 and C26.

1. Context and users (invented). StudyBuddy is an app for Class 10 board-exam students in smaller towns, in Hindi and English first. Users are students aged 14 to 16 on low-end Android phones (2 to 3 GB of memory) with intermittent data; the payer is a parent; the alternatives are a paid tuition class and free videos.

2. Discovery. *Evidence:* 24 student interviews, 12 parents and 8 teachers; a survey of 410 students found that 62% (about 254) had been stuck on a topic in the past week with nobody to ask; observation showed doubts cluster between 9 and 11 at night. When asked what they wanted, students said "more videos": a request for a solution (C29.3), not a description of the job. *Riskiest assumptions:* (1) students and parents will trust answers in their language that are tied to the textbook; (2) parents will pay ₹199 a month; (3) it can work on low-end phones and weak data. The first two are value risks and the highest in impact.

The cheapest test (FW-C26-05): a concierge test on a messaging channel for 40 students for four weeks: a language model drafts answers from the textbook, and a teacher reviews before sending. Kill-criteria card (FW-C30-03), written before the test:

Measure	Proceed if	Stop if	Middle zone
Questions per student per week	at least 3	under 1.5	persist with a better prompt to students
Teacher-reviewed answers correct (n = 300)	at least 90%	under 80%	build grounding and retest
Parents who pay ₹199 after two free weeks	at least 10%	under 5%	test a lower price

Result: 6.8 questions per student a week; of 300 reviewed answers, 273 were correct (91%), 18 partly correct (6%), 9 wrong (3%); 6 of 40 parents paid (15%). All proceed criteria were met, and the 9% not fully correct showed that accuracy needed work before scale.

3. Spec excerpt. *Problem statement:* "Class 10 students get stuck on topics at night and have no reliable help; they need a correct explanation in their language, tied to their textbook, on a low-end phone with unreliable data." *Goals:* at least 95% of answers correct on a 300-question evaluation set (with at most 1% wrong on a safety set); 95th-percentile response time of 6 seconds or less on a 3G connection; at most 3 MB of data per session; 3 doubts resolved per active student per week. *Non-goals:* predicting grades; replacing teachers; open-ended chat; medical or personal advice. *Requirements:* answers generated from retrieved textbook passages, with the page cited (C24.3); a fallback "I'm not sure; ask your teacher" when retrieval confidence is low; the last 50 answers cached for offline reading; verifiable parental consent for minors and no sale of data [V: DPDP Act 2023, section 9: a child is anyone under 18; verifiable parental consent is required and tracking, behavioral monitoring and targeted advertising directed at children are prohibited, with limited exemptions in the Rules] (India's data protection law has rules for children's data); no advertising. *Acceptance:* the evaluation set and latency and data limits above on a mid-range test device.

4. Metrics tree. North star: *doubts resolved per active student per week* = active students × sessions per student × doubts per session × resolution rate, where "resolved" means a thumbs-up or no repeat of the question within 24 hours. Business: paid conversion and contribution per active student. *Guardrails:* wrong-answer reports per 1,000 answers; teacher-flagged errors; data cost per session; parent complaints; share of sessions after midnight (wellbeing).

5. MVP, kill criteria and results. *MVP:* 400 students for 8 weeks with grounded answers. Kill criteria: stop if week-8 retention is under 15% or correctness under 90%; proceed above 25% and 95%. *Results:* 3.1 doubts resolved per active student per week; week-8 retention 31%; of 500 reviewed answers, 477 were correct (95.4%) and 8 wrong (1.6%, 95% interval about 0.5% to 2.7%); 95th-percentile latency 5.4 seconds; 2.1 MB per session; 18% of parents paid ₹199 (the threshold was 10%).

Economics (invented): 3.1 doubts a week is about 13.3 answers a month; at ₹0.60 per answer plus ₹6 hosting and data and ₹4 support, cost per active student is about ₹18 a month. With 18% paying ₹199, revenue per active student is about ₹35.80, contribution about ₹17.80 (about 50%).

6. Tradeoffs. (1) Grounding versus coverage: answering only from retrieved passages makes the product decline about 14% of questions (invented); accepted, because a wrong answer costs more trust than a refusal. (2) Two languages first versus five: accuracy and review capacity set the limit. (3) Offline cache versus freshness: the cache is small and safe for textbook content. (4) Pricing directly to parents versus school partnerships: start direct and test a school channel for lower acquisition cost.

7. Retrospective. *Pattern*: the first teacher-review sample came from chapters teachers liked, which were easy; correctness on a stratified sample by chapter difficulty was lower in the hardest chapter (88% on physics numericals, invented) than the overall 95.4%. The anti-pattern was a sample too convenient (C29.3, validating with the easy case). *Practice changed*: evaluation sets are stratified by difficulty, and the launch criterion applies to the weakest stratum, with special handling (a step-by-step solver with a check) for numericals. *Check*: the next evaluation reports the weakest stratum beside the average.

What this shows. Discovery named a value risk and trust risk, a manual test with thresholds de-risked them cheaply, the spec turned the evidence into requirements and non-goals, and the retrospective changed a measurement habit.

Worked example 2 — Journey 2: an enterprise operations tool (Illustrative)

Global; exercises C16, C17 and C27.

1. Context and users (invented). ExceptionDesk is an exception-management inbox for third-party logistics warehouses. Users are shift supervisors; the economic buyer is a vice president of operations; administrators and security sit in IT. Alternatives: spreadsheets, email and the screens of the warehouse management systems (WMS).

2. Discovery. *Evidence*: shadowing at 6 sites across 12 shifts and 14 interviews; a time study showed supervisors spend 2.4 hours of an 8-hour shift on exceptions (damaged, mismatched and late items), and 35% of that time (0.84 hours) goes to searching three systems for information. *Riskiest assumptions*: (1) a consolidated inbox cuts handling time by at least 30% (value); (2) integration with the three WMS vendors is feasible (feasibility); (3) buyers will pay \$1,800 per site per month and pass security review (viability). *Alpha tests*: a clickable prototype with 8 supervisors and an integration spike: the first WMS exposed events with about a 40-second lag, the second needed about 6 weeks of work, the third had no usable interface.

3. Spec excerpt. *Problem statement*: "Supervisors lose about an hour of each shift searching for information across systems; they need one place to see, prioritize and resolve exceptions." *Goals*: reduce exception handling time by 30% within 90 days of go-live at pilot sites; at least 70% of supervisors active weekly. *Non-goals*: replacing the WMS; automating resolutions; a mobile app in version 1. *Requirements*: events ingested within 60 seconds;

single sign-on; role-based access; an audit log; data kept in the customer's region; availability of 99.5%. *Acceptance*: handling time measured from event timestamps at 3 sites; security documentation ready for review (C27.2).

4. Metrics tree. Revenue = sites × price per site; sites = pipeline × win rate, with a sales cycle of about 7 months. Outcome: exceptions resolved per supervisor-hour. Adoption: weekly active supervisors ÷ supervisors. Retention: net revenue retention. *Guardrails*: misrouted exceptions, service-level breaches, support tickets per site.

5. MVP, kill criteria and results. *MVP*: 3 pilot sites for 90 days with the first WMS integration only. Kill-criteria card: proceed if handling time is down 25% or more by day 90 and weekly use is at least 70%; stop if the reduction is under 15% at day 60 or weekly use under 60%. *Results*: at day 60, a 22% reduction and 71% weekly use (middle zone: persist, fix onboarding); at day 90, handling time fell from 2.4 to 1.75 hours (27%), and weekly use was 78%. Two of three pilot sites (67%) converted to paid. A year later, a cohort of 20 customers with \$360,000 of annual recurring revenue had lost \$24,000 to churn and gained \$100,000 in expansion: net revenue retention of $(360 - 24 + 100) \div 360 = 121\%$.

6. Tradeoffs. (1) One integration first versus three: one gives a pilot in 12 weeks; three would have delayed every pilot by about six months. (2) Self-serve versus sales-led: security review and integration make sales-led the practical choice at first. (3) Configurable rules versus fixed workflow: configurable, because sites differ. (4) Price per site versus per exception: per site, for predictable bills.

7. Retrospective. *Pattern*: the roadmap as a promise (C29.2): early in the project the team told sales that three integrations would ship in the quarter; one shipped on time. A second miss: the security review took about 10 weeks longer than planned because the documentation was not ready (a non-feature requirement, C27.2). *Practice changed*: commit to outcomes and ranges ("first integration in 12 weeks; the next two sequenced by pilot demand"); put the security and compliance work in the plan at the end of discovery; keep a requirements checklist of non-feature items. *Check*: the share of committed items delivered on time, and the number of late-added requirements per deal.

What this shows. Staged integration, kill criteria with a middle zone and an honest account of a non-feature delay turned an enterprise launch into a repeatable process.

Worked example 3 — Journey 3: a marketplace with a cold start (Illustrative)

India-specific; exercises C19, C26 and C27.

1. Context and users (invented). RestockLink connects small neighborhood shops with wholesalers. A shop owner now orders stock two or three times a week by phone, often on 15 to 30 days of credit, from wholesalers whose price lists change daily. The first district has about 600 shops and 25 wholesalers.

2. Discovery. *Evidence*: 30 shop interviews, 12 wholesaler interviews and four days of shadowing deliveries; a survey of 120 shops showed that 41% had run out of a fast-selling item in the past week; wholesalers said about a third of their orders arrive by phone and are sometimes mis-recorded. *Riskiest assumptions*: (1) wholesalers will accept platform orders

and update price lists daily (supply behavior); (2) shops will order through a messaging-style flow in their language; (3) a 2% fee on order value can cover the cost of running the service. *Test (concierge)*: 30 shops and 6 wholesalers for 6 weeks; the team takes orders on a messaging app in the local language and phones wholesalers; shops pay the wholesaler directly and the team only records orders. Kill criteria, written first:

Measure	Proceed if	Stop if	Middle zone
Orders accepted by a wholesaler within 30 minutes	at least 60%	under 50%	fix price-list freshness and retest
Shops ordering at least 3 times in 6 weeks	at least 50%	under 40%	interview the lapsed shops
Concierge cost per order	under ₹120	over ₹200	find automation targets

3. Spec excerpt (MVP). *Problem statement*: "Shop owners lose sales to stockouts and time to phone ordering; wholesalers lose orders to mis-recording and cannot see demand." *Goals*: orders acknowledged within 30 minutes in at least 80% of cases; 60% of orders accepted within 30 minutes. *Non-goals*: owning logistics; lending; ranking wholesalers. *Requirements*: an order flow in the messaging channel in the local language; price lists uploaded by photo or sheet; delivery slots; payment on delivery or by instant bank transfer to the wholesaler; a credit ledger that only records what the wholesaler extends. *Regulatory note*: lending, holding customer funds and payment processing are regulated activities in India and need review before the product does more than record [V: RBI payment aggregator directions, 2025, for payments; lending rules not read]. *Acceptance*: the goals above in one district for four consecutive weeks.

4. Metrics tree. GMV = active shops × orders per shop per week × average order value. Liquidity (C27.1): share of orders accepted within 30 minutes; fill rate; repeat ordering. Supply: wholesalers with a price list updated in the last 24 hours. Revenue = GMV × fee. *Guardrails*: disputes per 100 orders, credit defaults, late deliveries.

5. MVP, kill criteria and results. *Results (6 weeks)*: 412 orders at an average of ₹4,500 (GMV ₹18.54 lakh, or ₹1,854,000); 62% of orders accepted within 30 minutes (about 255); fill rate of accepted lines 84%; 17 of 30 shops (57%) ordered at least three times. All proceed criteria were met on liquidity and repeat.

Economics (invented): a 2% fee is ₹90 per order (₹37,080 in total); concierge cost was ₹95 per order (₹39,140); credit defaults ran 0.4% of order value (about ₹18 per order, ₹7,416 in total). Contribution per order was $90 - 95 - 18 = -₹23$, a loss of ₹9,476 over the test. If automation brings the cost to ₹35 per order, contribution is $90 - 35 - 18 = ₹37$ per order.

6. Tradeoffs. (1) Depth in one district before a second: liquidity is local. (2) Supply first versus demand first: demand was ready; wholesalers' price lists were the bind, so supply quality (freshness) came first. (3) Ledger only versus lending: record only, until the regulatory review. (4) A messaging flow versus an app: the channel shops already use, with an app later.

7. Retrospective. *Pattern:* the early dashboard counted "wholesalers listed," a vanity count (C29.2), while the real constraint was whether orders were accepted within 30 minutes; the first month's number of listed wholesalers rose while acceptance stayed near 50%. *Practice changed:* the team reports accepted-within-30-minutes beside every supply count, asks wholesalers to confirm their price list each morning, and does not add shops until acceptance holds above 75%. *Check:* weekly acceptance rate and the share of wholesalers with a list updated in 24 hours.

What this shows. A manual start tested supply behavior and economics together, thresholds and a negative-contribution result pointed to automation targets, and the retrospective replaced a count with a liquidity measure.

Worked example 4 — Journey 4: an internal AI evaluation platform (Illustrative)

Global; exercises C21, C25 and C27.

1. Context and users (invented). EvalHub is an internal platform that lets product teams build evaluation sets, run them automatically on every prompt or model change, send outputs to human reviewers and calibrate model-based judges. There are 40 product teams building AI features, and their customers are other teams (C27.7); the platform cannot make them adopt it.

2. Discovery. *Evidence:* interviews with 14 teams: 6 had no evaluation set, 5 had sets of under 50 cases, 3 had larger sets and regression runs; four AI-feature rollbacks in the last half-year were attributed to untested prompt changes. *Riskiest assumptions:* (1) teams will adopt without a mandate (adoption risk); (2) shared rubrics can fit varied tasks (fit); (3) model-based judges can be trusted enough to use (reliability, C25.3). *Test:* an "evaluation in a notebook" template used by 3 teams. Kill criteria:

Measure	Proceed if	Stop if	Middle zone
Time to first evaluation run	under 1 day	over 3 days	simplify the template
Teams running evaluations weekly after 8 weeks	at least 3 of 5 pilot teams	under 2	interview non-adopters
Judge recall of bad outputs (200 labeled)	at least 75%	under 55%	add claim-level checks

3. Spec excerpt. *Problem statement:* "Teams change prompts and models without a reliable way to see what got worse; they need an evaluation they can set up in a day and trust."

Goals: time to first run under 1 day; a regression run on every prompt or model change; a judge offered to teams only after calibration against human labels. *Non-goals:* choosing models for teams; acting as a launch gatekeeper; storing production user data.

Requirements: dataset versioning; a human review queue; a calibration report (agreement and recall of bad outputs); cost tracking per run; per-team permissions; availability of 99%.

Acceptance: the three criteria above on the pilot teams.

4. Metrics tree. Value = teams onboarded × weekly regression runs per team × regressions caught that would have shipped. Adoption: teams running weekly. Reliability: availability, run time. Cost: per run. *Guardrails:* support load, team satisfaction, false alarms from judges.

5. MVP, kill criteria and results. *Pilot:* 5 teams for 8 weeks. *Results:* 4 of 5 teams ran evaluations weekly; time to first run was 1.6 days (above the 1-day goal, under the 3-day stop line: middle zone, simplify the template); 11 regressions were caught, 3 of which (27%) the teams said would have shipped; on 200 labeled outputs with 40 bad ones, the judge caught 25 (62.5% recall) at first and 31 (77.5%) after adding claim-level checks; cost was about \$4.20 per run.

Adoption plan: 12 of 40 teams (30%) adopted by the end of the first quarter; the goal was 25 teams (62.5%) by the end of the second, through templates, office hours and showing regressions caught, with no mandate except for launches classified high-risk (C25.4), where the risk function agreed to require evidence.

6. Tradeoffs. (1) Standard rubrics versus custom: standard templates for common tasks, custom fields for the rest. (2) Mandate versus value: value first; a narrow mandate where risk is high. (3) Build versus buy for the judging components: build the calibration and workflow, buy or reuse model access. (4) Central versus team-owned datasets: team-owned data in a shared system.

7. Retrospective. *Pattern:* the platform's early dashboard celebrated "evaluations run per week," a vanity metric (C29.2); some teams ran trivial evaluations to hit a target. *Practice changed:* the headline became regressions caught that would have shipped and coverage of known failure modes, with evaluation counts as a diagnostic; every quarter the team checks that the headline metric cannot be raised by trivial runs. *Check:* the share of evaluation sets with at least 100 cases covering listed failure modes.

What this shows. A platform with internal customers measured adoption and value, not activity, and calibrated its judge before offering it.

Worked example 5 — Journey 5: an autonomous agent for purchase requests (Illustrative)

Global; exercises C24, C25 and C32.

1. Context and users (invented). PurchaseFlow is an agent for a company's purchase requests. Employees submit requests by form or email; the agent classifies the request, finds approved vendors, checks policy, drafts a purchase order (PO) and routes it for approval. A procurement team of 12 handles about 4,000 requests a month.

2. Discovery. *Evidence:* 20 interviews and a process map; the baseline is a median of 9 days to a PO, and 38% of requests need a clarification round. *Shadow mode (C24, C25):* the agent ran silently on 600 historical requests. Accuracy by step: classification 97%, vendor match 92%, policy check 94%, PO fields 95%, approval routing 99%. Chained, the probability that all five steps are right is $0.97 \times 0.92 \times 0.94 \times 0.95 \times 0.99 =$ about 79%, too low for autonomy. *Riskiest assumptions:* (1) reliability across the chain; (2) approvers will review drafts carefully; (3) safety against hostile text in vendor emails and attachments (prompt injection, C25.4).

3. Spec excerpt. *Problem statement:* "Purchase requests take a median of 9 days because requests are incomplete and manual; a draft that is right most of the time and checked by a person can cut that." *Goals:* median days to PO under 6; at least 95% correct fields on a 500-case evaluation. *Non-goals:* autonomous purchasing in version 1; negotiating prices; paying invoices. *Requirements:* draft-only, with human approval for every PO; all external text treated as untrusted data and never as instructions; an allow-list of vendors and tools; a spending cap per request; an audit log of every step; low-confidence escalation to a person; a kill switch. *Acceptance:* the field-accuracy goal; zero successful attacks in 200 adversarial cases (which bounds the attack success rate at about 1.5% with 95% confidence); the audit log complete.

4. Metrics tree. Outcome: median days to PO; share of requests without a clarification round; procurement hours per request. *Guardrails:* wrong-PO rate; policy violations; injection incidents; approver override rate; a rubber-stamp indicator (approvals in under 5 seconds with no edits); cost per request.

5. MVP, kill criteria and results. *MVP:* draft-only for 120 users for 8 weeks. Kill criteria: stop if wrong-field rate exceeds 5% or approvers override more than 30%; proceed below 5% and below 25%. *Results:* of 500 reviewed drafts, 96.2% had all fields correct; the override rate was 18%; median days to PO fell from 9 to 5.5 (a 39% reduction); cost per request was about \$0.42 against about \$14 of handling cost.

The rubber-stamp test: the median approver spent 3.1 seconds on a draft. The team seeded 50 known errors into drafts; reviewers caught 30 (60%) and missed 40%. After changing the approval screen to highlight changed and low-confidence fields, the same test caught 41 of 50 (82%) (invented).

Earned autonomy (FW-C25-05): the first candidate for acting alone is low-value reorders under \$500 from an allow-listed vendor, after 300 supervised cases with zero errors (which bounds the error rate at about 1% with 95% confidence), with automatic demotion to draft-only if any error occurs.

6. Tradeoffs. (1) Autonomy versus control: draft-only first, autonomy by evidence for narrow bands. (2) Speed versus risk: approvals add time and catch errors; tune the screen so that review is fast and real. (3) Integrate with the ordering system now versus later: later, after reliability is shown. (4) Showing reasoning versus concise differences: concise differences, with reasoning on demand.

7. Retrospective. *Pattern:* the demo as evidence (C29.3): early sessions looked impressive in a scripted demo and the team nearly treated approval as proof of safety; the seeded-error test showed that approvals were mostly clicks. *Practice changed:* seeded-error tests run monthly as a standing check on reviewers; autonomy is granted per band after supervised evidence and withdrawn on a single error; and approval metrics include time and edits, not just acceptance. *Check:* the seeded-error catch rate each month and the number of bands promoted or demoted.

What this shows. A chain's reliability multiplied down, reviewers were tested and not assumed, and autonomy was a ladder earned by evidence.

6. Baseline, Strong, Standout

Prompt: "Walk me through a product you took from an idea to a launch."

Baseline answer

"I led a project to build a new feature. We talked to users, wrote requirements, built it over a few months and launched. It went well and users liked it."

What is wrong: no evidence, no assumption, no thresholds, no metrics, no learning.

Strong answer

"We saw that 40% of new users dropped at one step, so I interviewed ten users and found they didn't trust the data request. The riskiest assumption was that a clearer explanation would fix it, so we tested a prototype with 12 users and then an A/B test with 5% of traffic. I wrote a one-page spec with goals and non-goals, we launched in stages and completion went from 52% to 58%. I learned to test earlier."

Standout answer

Everything in the Strong answer, plus:

"I set the kill criteria before the test: proceed at a four-point gain, stop under one, with a guardrail on support contacts. The result was in the middle zone at week one and I extended the test rather than moving the threshold. The spec had two explicit non-goals, one of which (a redesign of the whole step) was requested by two stakeholders and declined with the data. At launch, completion rose six points and support contacts were flat. The retrospective found that my first interviews all came from active users; I now include lapsed users, and in the next project that changed the problem we solved. I can show the metric tree and the spec if useful."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
Evidence for the problem and a named riskiest assumption	Kill criteria set in advance and a rule for the middle zone
A staged test and a metric	Non-goals with the stakeholder pressure behind them
A lesson	A pattern, a changed practice and its use in the next project

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge [as of 2026-10].

Company lens: Google [as of 2026-10]. Candidates commonly report product-design and execution questions that ask for a user, a hypothesis and a launch plan with metrics Inferred . Practical consequence: tell the journey with a clear user and evidence, a hypothesis, a test and a decision rule.

Company lens: Meta [as of 2026-10]. Meta's published values emphasize moving fast and focusing on long-term impact Public [V: Meta's careers culture page, six core values] . Candidates commonly report execution questions that probe speed with evidence Inferred . Practical consequence: show the smallest test that could fail and the guardrail you used when you moved fast.

Company lens: Microsoft [as of 2026-10]. Microsoft sells to organizations and builds platforms, and candidates commonly report questions about customers, adoption and enterprise requirements Inferred . Practical consequence: use journeys 2 and 4 as models: the buyer, the user, the administrator and the non-feature requirements.

Company lens: Amazon [as of 2026-10]. Amazon publishes a principle on customer obsession and describes working backwards from the customer Public [V: Amazon's Leadership Principles; working-backwards material] . Candidates commonly report detailed probing of a project's numbers Inferred . Practical consequence: have the numbers for each stage and the customer evidence behind the decision.

Company lens: Apple [as of 2026-10]. Apple does not publish its development process, and candidates commonly report craft and judgment questions Inferred [unverified: candidate-reported] . Practical consequence: tell what you left out (non-goals) and why, without disclosing confidential details of past work.

Company lens: Netflix [as of 2026-10]. Netflix describes a culture of candor and context and widely uses experiments Public [V: Netflix culture memo; Netflix Tech Blog] . Candidates commonly report questions on testing and decision rules Inferred . Practical consequence: lead with how the test and the decision rule worked, including when the result was ambiguous.

Company lens: Stripe [as of 2026-10]. Stripe is widely described as valuing clear writing and rigor Inferred [V: The Pragmatic Engineer, Inside Stripe's Engineering Culture Part 2 (strong culture of writing); Slab interview with Stripe's former Documentation Manager]. Candidates report questions on spec quality and reliability Inferred. Practical consequence: bring the spec excerpt, with the acceptance criteria and non-goals, as an artifact.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	A list of activities, not decisions	"We did research, then design, then build"	Name each decision and its evidence
2	A spec written after the build	Requirements match what was built exactly	Write the spec from the evidence before building
3	Metrics chosen after results	Thresholds that fit the outcome	Write the kill-criteria card first
4	No middle zone	Only pass or fail	Decide what a mixed result triggers
5	No non-goals	Everything is in scope	Name what you will not do and why
6	The launch as the finish line	No review after launch	Success criteria and a review date
7	A memorized journey	Collapses at the second probe	Use your own project; know the numbers
8	A tidy story that hides the mess	No false starts or reversals	Say where you were wrong and what you changed
9	Ignoring non-feature requirements	Security, consent, regulation appear late	Put them in the plan at the end of discovery
10	A retrospective of virtues	"I learned to communicate"	A pattern, a practice and a check

9. Mental models applied

Four models, each tied to a decision in this chapter. [[inversion]] is from C02.

Staged risk reduction (MM-staged-risk). Spend the least on the most uncertain assumption first and increase spend as risk falls: a manual test before a build, a pilot before a launch, draft-only before autonomy. Each phase gate (FW-C30-02) is a decision to spend more.

Multiplication of reliability (MM-chain-reliability). When steps depend on each other, probabilities multiply: five steps at 92% to 99% give about 79%. Check each step and the chain, and put the human check where the product of errors is largest.

Inversion (MM-inversion, see [[inversion]], C02). To plan a journey, ask how it would fail: wrong problem, wrong sample, a metric that can be gamed, a review that is a click. Design a check for each.

Pre-commitment (MM-pre-commitment). Writing the rule before seeing the result protects against moving the goalposts [V: Nosek, Ebersole, DeHaven and Mellor, The Preregistration Revolution, PNAS 115(11), 2018, pp. 2600 to 2606: define the plan before observing outcomes]. Kill-criteria cards are pre-commitment devices.

Engineering, design and data-science perspectives

- **Engineering.** Engineers want the spec before the build, with non-goals and acceptance criteria, and they will tell you which assumption is technically riskiest: ask them at the end of discovery. They dislike scope that grows without a tradeoff.
- **Design and research.** Designers and researchers lead discovery and the prototype tests; involve them in choosing the riskiest assumption and the sample. A stratified sample (journey 1) is their kind of correction.
- **Data science.** Data scientists help set thresholds, sample sizes and intervals and review the metric tree; they dislike thresholds set after the data. Agree the kill-criteria card with them before the test.
- **Legal, security and operations.** They own the non-feature requirements (consent, security review, regulation, reliability); engage them at the end of discovery, not at launch.

Checklist: before you tell or plan a product journey

1. Is the problem supported by evidence, with numbers?
2. Have I named the riskiest assumption and why it was riskiest?
3. Was the test the cheapest that could fail, with thresholds and a middle zone written first?
4. Does the spec have a problem statement, goals, non-goals and acceptance criteria?
5. Is there a metric tree with a primary measure and guardrails?
6. Did I decide by the pre-set rule, and say how I used judgment in the middle zone?
7. Did I include non-feature requirements early?
8. Does my retrospective name a pattern, a practice and a check?

10. Case studies

CS-C30-01 · The memorized journey · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A candidate preparing for a product interview learns Journey 2 from this chapter nearly word for word and, in a mock, tells it as if it were their own project. The interviewer asks about the time study, the third integration and how the security review was handled.

Constraints. Two weeks to prepare; one real project that the candidate considers dull; a mock interviewer who probes for detail.

Data. In the mock the candidate answers the first two probes by repeating the journey, fails the third (the candidate cannot say who did the time study or what changed), and the interviewer asks six follow-up questions: three are answered with specifics, two with generalities and one with a figure that contradicts the earlier one. The mock scorecard rates evidence as weak and credibility as low.

Problem definition. As an observable problem: "The candidate told an invented story they did not own, and could not answer the second layer of questions."

Analysis. The journeys are teaching devices (C30.1); the habits they show (a riskiest assumption, thresholds set first, non-goals, a retrospective) transfer, and the content does not. The candidate's real project had the same stages, with a smaller scale, and contained facts the candidate knew in detail: a funnel drop of 38% at one step, ten interviews and a test.

Options.

Option	Optimizes	Costs
A. Keep telling the memorized journey	Fluency	Collapses under probing; risks dishonesty
B. Rebuild the answer from the candidate's own project with the journey template (FW-C30-01), with real numbers and one honest mess	Credibility and depth	Preparation time; a smaller-looking story
C. Use a hypothetical, labeled as such	Honesty with no real project	Less proof of past impact

Tradeoffs. B costs a few hours and survives probing; A is faster and fails the second question; C is acceptable when there is no real project and is labeled.

Decision. Choose B. Fill in the template with the real project, with the real numbers and one thing that went wrong and the practice changed; rehearse the probes ("how did you know that?", "what would have made you stop?").

Outcome. In this fictional scenario, the rebuilt answer used a smaller project (a funnel drop of 38%, ten interviews, a 12-user prototype test, a 5% A/B test) and the follow-ups were answered with specifics in 7 of 8 cases; the scorecard rated evidence as strong (invented).

Alternative outcomes. If the candidate's real project had been too thin, a clearly labeled hypothetical built on the template, plus a real smaller story about one stage, would have been honest and useful. The lesson: use the journeys to learn the habits, then tell your own story.

CS-C30-02 · A public phase-gate model: discovery, alpha, beta and live in UK government digital services · Verified · Advanced

Sources (types): the GOV.UK Service Standard page and Government Digital Service (GDS) blog posts of January 2014, June 2014 and September 2017 and the 2020 post on assessments after launch (primary); an archived page of the Government Service Design Manual in a government code repository (primary); a UK department's service-assessment page and its April 2026 blog post (primary); a New South Wales government toolkit page and a Scottish government page that restate the phases (primary, other governments). A consultant's history of the Service Standard gave 25 points for the first version, which the GDS posts contradict (26), and was not used. The GOV.UK service manual's own phase pages were not opened; the phases are read from the department and state pages. The analysis is the author's. Re-read in October 2026.

Situation. The UK Government Digital Service (GDS) publishes a Service Standard and a service manual to help teams build public digital services. The standard that GOV.UK lists today has 14 points [V: GOV.UK Service Standard page]. Its first version, the Digital by Default Service Standard, had 26 criteria and formally came into force in April 2014, when the government committed that all new or redesigned digital services would meet it before going live; GDS assessed the major services and departments assessed the smaller ones at each stage (alpha, beta and live) [V: GDS blog posts, January and June 2014]. GDS cut the 26 points to 18 in 2015 [V: GDS blog, 2017] and the standard was later reduced to the 14 points listed today. The delivery model has four phases: discovery, alpha, beta and live [V: a department's service-assessment page; state and Scottish government pages].

Discovery is about understanding users and their needs, constraints and the problem space; *alpha* is for testing different solutions to the problems found in discovery; *beta* is for building the best idea from alpha, starting with a private beta for limited users and moving to a public beta open to anyone; *live* is for supporting and continuing to improve the service [V: department's service-assessment page]. Formal assessments occur at the end of alpha, between private and public beta, and before live, with an informal peer review at the end of discovery [V: same; GDS blog, 2020]. A GDS post of 2020 notes a gap: after the live assessment there was no planned formal engagement with service teams, although the manual expects continuous improvement in the live phase [V: GDS blog, 2020]. A department added a fifteenth point on sustainability for its own assessments in April 2026 [V: department blog, 2026].

Constraints. Many departments with different capabilities building many services; public money and a need for services to be accessible to everyone; a risk of large, late projects that deliver the wrong thing; and a need for an assurance process that fits agile work and does not become a bureaucracy.

Data. As reported: the phases and what each is for; where assessments occur; the changes to the standard's number of points; the live-phase gap noted by GDS; and adoption of the same phases elsewhere.

Problem definition. *The author's reading.* The model addresses an old problem in large projects: committing the whole budget before the riskiest assumptions are tested. As an observable problem: "How to stage spending and scrutiny so that evidence, not a plan, earns the next phase?"

Analysis. *The author's analysis, through this chapter's tools.*

It is staged risk reduction (MM-staged-risk). Discovery spends little to understand the problem; alpha tests solutions to the riskiest assumptions with prototypes; private beta tests with real users in a limited setting; public beta scales; each assessment is a gate where the team shows evidence from the previous phase before getting permission and funding for the next. This is the structure of every journey in this chapter, and of FW-C30-02.

The gates demand evidence, not plans. An archived page of the Government Service Design Manual says that to pass the beta assessment a team must explain how it has built on its work and evidence from the discovery, alpha and private beta phases [V: archived government manual page]. That is the equivalent of the kill-criteria card: criteria stated, evidence shown, decision made.

Where it can fail [Inf]. A gate can become a form to complete; the assessment can be passed without the team believing in the evidence; and the live phase, where most of a service's life is spent, has had the least formal review (the 2020 GDS post acknowledges this). The same weakness appears in product teams that celebrate a launch and stop measuring (C29.4).

What the evidence does not show. How many services were stopped or changed at a gate, the cost savings, or how consistent assessors were. The case supports an analysis of the phase-gate structure and its limits, not a claim that it guarantees good services.

Options. *The author's reconstruction of choices a central digital team could face; not a record of its deliberations.*

Option	Optimizes	Costs
A. Approve whole projects up front on a business case	Predictability of spend	Commits before evidence; large late failures
B. Fund in phases with assessments at gates and a published standard	Evidence before spend; learning; consistency	Process overhead; the risk of box-ticking
C. Leave it to each department	Autonomy	Inconsistent quality and accessibility

Tradeoffs. B costs process and delivers evidence-led funding; A is simpler and bets more on a plan; C is flexible and uneven.

Decision. As documented, the central digital team published a phased model and a standard, and the government committed that new or redesigned digital services would meet the standard before going live [V: GDS blog posts, January and June 2014].

Outcome. Reported: the model remains in use, a department added a point to the standard in 2026 [V: department blog], and other governments describe the same phases (New South Wales and Scotland) [V: state and Scottish government pages]. Effects on delivery outcomes are not described in the sources read.

Alternative outcomes. *Inference.* A model with the gates but no exit criteria would be a calendar; a model with criteria but no live-phase review would repeat the launch-as-finish-line anti-pattern. A reader should avoid hindsight and uncritical admiration: a public process earns trust by publishing its standard, and its value depends on how it is used. The method to carry away: stage spend by evidence, write exit criteria before each phase, and keep reviewing after launch.

11. Interview questions and model answers

Eight questions. Model answers are *Illustrative* (composite candidates), and every figure is invented (invented). Each ends with follow-ups and answers. No company tag is attached because the topic is not tied to a single company.

IQ-C30-01 · Beginner · execution · EX

Prompt. How would you take a new product idea from discovery to launch?

Rubric.

- **Baseline:** "Research, design, build, launch."
- **Strong:** Stages with evidence and a test before the build.
- **Standout:** Names the riskiest assumption, a cheap test with thresholds set first, a spec with non-goals, a metric tree with guardrails, a staged rollout and a review after launch.

Model answer. "I'd stage it so that spend follows evidence. Discovery: understand the user and the problem with interviews, observation and data, and list the assumptions; the riskiest is usually that people will pay or change behavior. Alpha: test that assumption with the cheapest experiment that can fail, such as a smoke test or a manual version, with the thresholds written first (proceed, stop, and what a middle result triggers). Spec: a short document with the problem statement, goals, non-goals and acceptance criteria. Metrics: a tree with a primary measure, the drivers and a guardrail. Beta: a staged rollout, private then wider, with a rollback trigger. Live: a review at a set date against the success criteria, and a retrospective that records the pattern and what I'd change.

"At each gate I'd decide to proceed, persist, pivot or stop by the rules written before the evidence arrived."

Signals. Evidence-gated stages; thresholds first; non-goals; a review.

Follow-ups.

- **Q.** What if you have no time for alpha? **A.** "Then I'd shrink it: a day of customer calls and a fake-door test can test the riskiest assumption cheaply. Skipping it entirely means committing before evidence; I'd say so and set a kill criterion for the first release."

- **Q.** How do you decide the thresholds? **A.** "From the economics and the cost of being wrong: what conversion or retention makes the product worthwhile, and a stop line below which it isn't, with a middle zone for a better test. I agree them with the data scientist and the stakeholders before the test."

IQ-C30-02 · Beginner · execution · EX

Prompt. What is the difference between discovery, alpha, beta and live?

Rubric.

- **Baseline:** "Early, middle and late stages."
- **Strong:** The purpose of each.
- **Standout:** States the evidence needed to exit each phase, the decision at each gate and that the live phase keeps reviewing; notes that public-sector digital teams use this model.

Model answer. "Discovery is for understanding users, constraints and the problem; its exit is evidence of the problem, the riskiest assumptions and a plan for testing them. Alpha is for testing different solutions to those assumptions, usually with prototypes; its exit is a tested solution with results against thresholds. Beta builds the best option and tests it with real users, first a limited private group and then everyone; its exit is a working end-to-end service with metrics, guardrails and operational readiness. Live runs and keeps improving the service, with reviews of outcomes and a plan for the next cycle. Public-sector digital teams, such as the UK government's, publish this four-phase model with assessments at gates, and it maps well onto product work."

Signals. Purpose and exit evidence for each; gates; continued review.

Follow-ups.

- **Q.** Can a phase be skipped? **A.** "A small, reversible change might combine them, but skipping alpha on a large bet commits spend before the riskiest assumption is tested. I'd compress, not skip, and record what I did not test."
- **Q.** What happens after live? **A.** "The same loop continues for smaller pieces of work: new discovery on the next problem, tests, releases. The common failure is stopping the review after launch, so I put a review date in the plan."

IQ-C30-03 · Intermediate · product-sense · PS

Prompt. You are building a learning app for students in a market with low-end phones and weak data. What goes into the first version?

Rubric.

- **Baseline:** "Videos and quizzes."
- **Strong:** Starts from the job and constraints.
- **Standout:** Names the riskiest assumptions (trust and willingness to pay), tests them manually first, designs for data and device limits, grounds AI answers in the textbook, and sets accuracy criteria by stratum.

Model answer. "I'd start from the job: students stuck on a topic with no one to ask at night. The riskiest assumptions are that they and their parents will trust the answers and that parents will pay. I'd test that manually before building: a concierge test on a messaging channel for 40 students for four weeks, with drafts reviewed by a teacher, and thresholds written first (for example, at least 90% of reviewed answers correct, at least 3 questions per student per week, at least 10% of parents paying after two free weeks).

"First version: answers generated from the textbook passages with the page cited, a fallback to 'ask your teacher' when unsure, offline caching of the last answers, under 3 MB of data a session, and a 95th-percentile response under six seconds on a slow connection. Not in version one: videos, gamification, grade prediction. Accuracy criteria by difficulty stratum, not just an average, since an average can hide a weak chapter. Guardrails: wrong-answer reports, data cost, parent complaints, late-night use."

Signals. Job first; riskiest assumptions; manual test; device constraints; stratified accuracy.

Follow-ups.

- **Q.** Why not start with the videos students asked for? **A.** "They asked for a solution; the job is getting unstuck. Videos are expensive and don't resolve a specific doubt at night. I'd test what resolves doubts fastest with the concierge version, and add video if the data shows it."
- **Q.** What if accuracy is 91% in the test? **A.** "That is not enough to scale. I'd ground answers in the textbook, add a fallback, and retest with a stratified sample until the weakest stratum meets the bar; and I'd tell users what the tool can't do."

IQ-C30-04 · Intermediate · strategy-gtm · ST

Prompt. How would you start a marketplace connecting small neighborhood shops with wholesalers?

Rubric.

- **Baseline:** "Build an app and onboard both sides."
- **Strong:** Starts in one district and tests manually.
- **Standout:** Names the supply behavior as the riskiest assumption, measures liquidity (acceptance within a time, fill rate, repeat), computes concierge economics, and notes regulatory limits on credit and payments.

Model answer. "I'd start in one district and run it by hand. The riskiest assumption is supply behavior: will wholesalers accept orders through a platform and keep price lists current? I'd take 30 shops' orders on a messaging app in their language and phone wholesalers myself for six weeks, with thresholds written first: at least 60% of orders accepted within 30 minutes (stop under 50%), at least half the shops ordering three times (stop under 40%).

"I'd measure liquidity, not sign-ups: the share of orders accepted within 30 minutes, the fill rate and repeat ordering, with disputes and credit defaults as guardrails. Economics: with a 2% fee on a ₹4,500 order (₹90) against ₹95 of concierge cost and ₹18 of default cost, contribution is negative ₹23 an order, so the automation targets are the manual coordination

steps: bringing cost under about ₹35 makes it about ₹37 positive. I'd keep credit to a ledger that only records what wholesalers extend until I've checked the rules on lending and payments, and stay in the district until acceptance holds above 75%."

Signals. One district; liquidity measures; economics; regulatory caution.

Follow-ups.

- **Q.** What if wholesalers won't update price lists daily? **A.** "That would be the finding that matters. I'd test lower-effort ways (photo of the list, a morning confirmation prompt), and if they won't, shops can't get reliable prices and the marketplace doesn't work as designed; I'd pivot to a different supply arrangement."
- **Q.** How do you expand? **A.** "To a second district only when acceptance and repeat hold, using what the manual work showed about which steps to automate, and repeating the concierge start there with fewer coordinators."

IQ-C30-05 · Intermediate · strategy-gtm · ST

Prompt. Your enterprise pilot succeeded at three sites. Why might the deal still stall, and what do you do?

Rubric.

- **Baseline:** "The customer might not have budget."
- **Strong:** Names the buyer, security and integration.
- **Standout:** Maps the buying committee, plans the non-feature requirements early, plans for a seven-month sales cycle, prices to the buyer's logic, and sets the pilot's exit criteria to include procurement and security.

Model answer. "A pilot proves the users like it; the deal needs the economic buyer, security, IT and procurement. It stalls when a security review or a single sign-on or audit-log requirement appears late, when the champion lacks budget authority, or when integration beyond the pilot system is needed. In one project a security review added about ten weeks because the documentation wasn't ready."

"So I'd map the buying committee at the start of the pilot (economic buyer, champion, administrators, IT and security, procurement and legal), put the security and compliance work into the plan at the end of discovery, write exit criteria that include procurement and security milestones, and give the buyer a business case in their terms: handling time down 27%, about 0.7 hours a shift saved per supervisor, against \$1,800 per site per month (the figures here are from Journey 2 in C30, all invented). I'd plan for the real sales cycle (about seven months) and commit to outcomes and ranges to sales, not dated features."

Signals. Buying committee; non-feature requirements early; a business case; realistic cycle.

Follow-ups.

- **Q.** What if two of three pilot sites convert and one does not? **A.** "I'd learn why: the site's fit, the supervisor adoption or an integration limit. A third that doesn't convert may show a segment where the product doesn't fit, which is useful for targeting."

- **Q.** How would you price it? **A.** "Per site, for predictable bills and a simple sale, with expansion by site count; I'd test per-exception pricing only if usage varies a lot between sites."

IQ-C30-06 · Hard · ai · AI

Prompt. An agent drafts purchase orders and a person approves each one. The median approval takes three seconds. Is that safe?

Rubric.

- **Baseline:** "Yes, a person approves everything."
- **Strong:** Questions whether reviewers really check.
- **Standout:** Tests reviewers with seeded errors, redesigns the approval screen to surface changes and uncertainty, tracks time and edits, and ties autonomy to supervised evidence by band with demotion.

Model answer. "Not necessarily: approval time of three seconds suggests many approvals are clicks, and a human in the loop only helps if the human is checking. I'd test it: seed a known set of errors into drafts, say 50, and see how many reviewers catch. In one test they caught 30 of 50 (60%); after changing the screen to highlight changed and low-confidence fields, they caught 41 of 50 (82%). I'd run that test monthly as a standing check."

"I'd also track approval time, edits and override rates, not only acceptance. For autonomy, I'd use a ladder: draft-only for everything first; then, for a narrow band such as low-value reorders from an allow-listed vendor, autonomy after 300 supervised cases with zero errors (which bounds the error rate near 1%), and automatic demotion to draft-only on any error. Controls around it: untrusted-text handling against prompt injection, spending caps, an audit log and a kill switch."

Signals. Reviewer testing; screen design; time and edits; earned autonomy; controls.

Follow-ups.

- **Q.** Won't the extra review slow everything? **A.** "A better screen makes review faster and more real: reviewers look at the three fields that changed, not the whole document. I'd measure review time and catch rate together."
- **Q.** What would make you promote a second band? **A.** "The same evidence standard: a supervised run with zero errors at the required sample, a clear definition of the band, and a demotion rule. I'd promote slowly and tell finance and procurement what changed."

IQ-C30-07 · Hard · ai · AI

Prompt. You run an internal AI evaluation platform used by 40 product teams. You cannot mandate adoption. How do you drive it, and how do you measure success?

Rubric.

- **Baseline:** "Promote it and track how many teams use it."
- **Strong:** Reduces friction and measures value.

- **Standout:** Treats teams as customers, measures regressions caught rather than activity, keeps time to first run low, calibrates judges, uses narrow mandates only where risk is high, and checks that the metric can't be gamed.

Model answer. "I'd treat the teams as customers. Interview non-adopters to find the barrier; in one case it was setup time, so I'd cut time to first evaluation run to under a day with a template. I'd show value with results: in an 8-week pilot with 5 teams, 4 ran evaluations weekly and caught 11 regressions, 3 of which would have shipped. I'd calibrate any model-based judge against human labels before offering it (for example, 62.5% recall of bad outputs rose to 77.5% with claim-level checks).

"For measures, the headline is regressions caught that would have shipped and coverage of known failure modes, not evaluations run, since teams can inflate that with trivial runs. Adoption goes from 12 of 40 to 25 through templates, office hours and visible wins, with no mandate except for high-risk launches where the risk function requires evidence. Guardrails: reliability, cost per run, support load and team satisfaction. Each quarter I'd check that the headline can't be raised by trivial runs."

Signals. Customer view; value measures; calibrated judge; narrow mandate; anti-gaming check.

Follow-ups.

- **Q.** What if a big team refuses? **A.** "I'd learn why and fix what's fixable, show them the regressions found elsewhere and offer a joint pilot. If they are in a high-risk category, the risk process can require evidence; otherwise their choice stands and I'd keep the door open."
- **Q.** How do you decide build versus buy for components? **A.** "Buy what is commodity (model access, storage), build what differs for us (workflow, calibration, permissions), and revisit when the market or our needs change."

IQ-C30-08 · Hard · behavioral · BE

Prompt. Tell me about a product you took from idea to launch.

Rubric.

- **Baseline:** A chronological description with no numbers.
- **Strong:** Evidence, a test and a result.
- **Standout:** A through-line with a riskiest assumption, thresholds set first, a spec with non-goals, a decision by the pre-set rule, and a retrospective with a pattern and a changed practice.

Model answer. (*A composite example; the figures are invented.*) "We saw that 38% of new users dropped at a permissions step. I interviewed ten users and found they didn't understand why we needed access, so the riskiest assumption was that a clearer explanation would fix it. We tested a prototype with 12 users, then an A/B test on 5% of traffic with

thresholds written first: proceed at a four-point completion gain, stop under one, and a guardrail on support contacts. The spec had two non-goals: no redesign of the whole flow, and no change to the permissions themselves.

"At week one the result was in the middle zone (a 2.5-point gain), so I extended the test two weeks rather than moving the threshold, and it reached 4.6 points. We launched in stages; completion rose from 52% to 58% and support contacts were flat. The retrospective found that my first interviews were all active users, which could have hidden why lapsed users never returned; I now include lapsed users in discovery, and in the next project that changed the problem we picked. I can show the metric tree and the spec."

Signals. Evidence; riskiest assumption; pre-set thresholds; non-goals; pattern and changed practice.

Follow-ups.

- **Q.** What would you have done if the test had been under one point? **A.** "Stopped and told the stakeholders what we learned: that explanation wasn't the barrier. I'd have gone back to the interviews to find the real reason, and recorded the result."
- **Q.** Who disagreed, and how did you handle it? **A.** "Two stakeholders wanted the whole step redesigned. I showed the funnel data (the drop was at one screen), agreed the test as the way to decide, and committed to revisit if the gain was small. They accepted because the rule was written first."

12. Exercises

Eight exercises. Each has a model solution and an alternative. Every figure is invented.

EX-C30-01 · A kill-criteria card

Problem. You will test a service that sends a weekly meal plan to busy parents by text message, run by hand for 30 families for four weeks. Write the kill-criteria card (FW-C30-03) with three measures, thresholds, sample, date and a middle-zone decision.

Context. Alpha testing.

Data. None.

Constraints. Seven minutes.

Expected approach. FW-C30-03 and C26.6.

Solution. (1) *Families who still use the plan in week 4:* proceed at 50% or more (15 of 30), stop under 25% (7 of 30); middle zone: interview the lapsed families and revise the plan format for a second month. (2) *Families who pay \$5 a week after two free weeks:* proceed at 20% or more (6 of 30), stop under 7% (2 of 30); middle zone: test a lower price. (3) *Delivery cost per family-week:* proceed if under \$3, stop if over \$8; middle zone: list automation targets. Sample: 30 families, with a note that 30 gives wide intervals, so a result near a threshold is treated as middle zone. Date: written before week 1 and dated; review at week 4. Decision owner: the PM with the data partner.

Alternative solution. Add a guardrail: complaints per family under 0.2 a week.

Tradeoffs. Small samples give rough answers; set thresholds with wide middle zones.

EX-C30-02 · Chain reliability

Problem. An agent has four steps with accuracies 96%, 90%, 95% and 98%. Compute the chance that all four are right. If a human reviewer catches 80% of errors before action, what is the residual error rate? Where would you put the check, and what would you test about the reviewer?

Context. Agents and approvals.

Data. As above.

Constraints. Six minutes.

Expected approach. MM-chain-reliability and Worked example 5.

Solution. All four right: $0.96 \times 0.90 \times 0.95 \times 0.98 =$ about 80.4%; chain error 19.6%. With a reviewer who catches 80%: residual error $19.6\% \times 0.2 =$ about 3.9% (96.1% correct). Put the check after the weakest step (90%) if the steps can be reviewed separately, or at the end where the product of errors is largest and an error becomes an action. Test the reviewer: seed known errors and measure the catch rate; the 80% is an assumption until measured, and a three-second review suggests it may be lower.

Alternative solution. Compute the residual if the reviewer catches 60%: $19.6\% \times 0.4 =$ 7.8%.

Tradeoffs. More checks reduce errors and add time and cost; place them where an error costs the most.

EX-C30-03 · Concierge economics

Problem. A manual ordering service charges a 2.5% fee on a ₹3,000 average order. Concierge cost is ₹80 per order; credit defaults run 0.5% of order value. Compute contribution per order. What cost per order gives a contribution of ₹30?

Context. Economics of a manual start.

Data. As above.

Constraints. Five minutes.

Expected approach. Worked example 3.

Solution. Fee: $0.025 \times 3,000 =$ ₹75. Defaults: $0.005 \times 3,000 =$ ₹15. Contribution: $75 - 80 - 15 = -₹20$ per order. For ₹30 of contribution: $75 - 15 - \text{cost} = 30$, so $\text{cost} = ₹30$ per order. Automating the manual coordination to ₹30 is the target; check whether it is feasible, and whether the fee or default rate could change instead.

Alternative solution. Raise the fee to 3.5% (₹105): contribution at ₹80 cost is $105 - 80 - 15 = ₹10$; test whether shops and wholesalers accept it.

Tradeoffs. A manual start loses money by design; the question is whether the path to a positive margin is credible.

EX-C30-04 · Unit economics of an AI learning feature

Problem. Students resolve 4.0 doubts a week; a month is 4.3 weeks. Cost per answer is ₹0.50; fixed cost per active student is ₹9 a month. 12% of active students pay ₹149 a month. Compute monthly cost per active student, revenue per active student and contribution. What conversion breaks even, and what do you do?

Context. Consumer AI economics.

Data. As above.

Constraints. Seven minutes.

Expected approach. Worked example 1 and C20.7.

Solution. Answers a month: $4.0 \times 4.3 = 17.2$; variable cost: $17.2 \times 0.50 = ₹8.60$; total cost: $8.60 + 9 = ₹17.60$. Revenue per active student: $0.12 \times 149 = ₹17.88$. Contribution: ₹0.28, about break-even. Break-even conversion: $17.60 \div 149 = 11.8\%$. At 15% conversion, contribution is $22.35 - 17.60 = ₹4.75$. What to do: raise conversion (stronger free-to-paid trigger, a school channel), lower cost per answer (caching common questions), or raise price after testing; check the sensitivity before scaling acquisition.

Alternative solution. Cache the 30% most common answers and compute the new variable cost (₹6.02 at a 30% cache hit rate).

Tradeoffs. Heavier use raises cost per active student; light use lowers value; price to the middle.

EX-C30-05 · A spec excerpt

Problem. Write a spec excerpt (FW-C30-01 part 3) for a feature that summarizes customer support tickets for agents: a problem statement, three goals, three non-goals, four requirements and acceptance criteria.

Context. Spec practice.

Data. None.

Constraints. Ten minutes.

Expected approach. C16 and the spec excerpts in section 5.

Solution. *Problem:* "Agents spend about four minutes reading a long ticket history before replying; they need a short, accurate summary." *Goals:* reduce time to first reply by 20%; at least 95% of summaries free of invented facts on a 300-ticket evaluation; at least 70% of agents using summaries weekly. *Non-goals:* sending replies automatically; summarizing across customers; storing summaries beyond the ticket's retention period. *Requirements:* summaries cite the ticket messages they draw from; the agent can edit or dismiss; personal data follows the existing retention and access rules; latency under 3 seconds for the 95th

percentile. *Acceptance*: the evaluation set (with a check on invented facts), the latency and a pilot with two agent teams meeting the goals for four weeks; a rollback if the invented-fact rate exceeds 2% in production.

Alternative solution. Add a guardrail: first-reply quality score must not fall.

Tradeoffs. Non-goals disappoint some requesters and keep the project deliverable.

EX-C30-06 · A metric tree for an enterprise tool

Problem. Draw a metric tree for an internal tool that helps sales staff find answers in company documents: a primary measure, three drivers, two guardrails, and the first driver you would examine if usage were high and satisfaction low.

Context. Metrics practice.

Data. None.

Constraints. Six minutes.

Expected approach. FW-C05-01 and C27.10.

Solution. *Primary*: questions answered correctly per salesperson per week (correct means the user accepted the answer and did not re-ask within a day). *Drivers*: active salespeople × questions per person × answer success rate; success rate depends on retrieval quality, document freshness and answer accuracy. *Guardrails*: wrong-answer reports per 1,000 and time to find an answer (latency). If usage is high and satisfaction low, examine answer success rate by question type first: usage shows people try it, so a failure in accuracy or freshness for common questions would explain low satisfaction; then check the share of "no answer" responses.

Alternative solution. Add a trust measure: the share of answers where users open the cited source.

Tradeoffs. A correct-answer measure needs a way to judge correctness; sample reviewed answers weekly.

EX-C30-07 · A retrospective with the anti-pattern scan

Problem. A team launched a feature on time. After six weeks, the metric was flat. Using the failure-story structure (FW-C29-03) and the catalog (C29), write a retrospective naming a pattern, the root cause in your control, the missed signal and a practice with a check. Invent two numbers and mark them (invented).

Context. Closing a journey.

Data. None.

Constraints. Eight minutes.

Expected approach. FW-C29-03 and C29.4.

Solution. "The pattern was *shipped counted as success*: we hit the date and declared it done. The root cause in my control: I never wrote a success criterion or a review date. The signal I missed: adoption among the target segment was 4% after two weeks against a 15% goal (invented), visible on the dashboard but not on anyone's review list. Practice changed: every launch has a metric, a target and a review date written before the build; a four-week review is on the calendar at kickoff. Evidence: the next two launches were reviewed on time, and one was stopped after the review because adoption stayed under 5%, saving about three engineer-weeks (invented)."

Alternative solution. Add a gate: no launch review in the calendar, no launch approval.

Tradeoffs. Reviews take time; a stopped project saves more than the review costs.

EX-C30-08 · Apply the journey template to your own idea

Problem. Fill in the journey template (FW-C30-01) for a real product idea or a product you shipped, with real numbers, thresholds, a spec excerpt, a metric tree and a retrospective. Then give it aloud in 30 minutes to a partner who probes.

Context. Practice for the idea-to-launch question.

Data. Your work.

Constraints. Two hours for the template; 30 minutes aloud.

Expected approach. FW-C30-01 and section 4.

Solution. A good template has: a context with a named user and alternative; discovery with at least two sources of evidence and a named riskiest assumption; a test with a kill-criteria card written before the result and a middle zone; a spec excerpt with a problem statement, goals, non-goals and acceptance criteria; a metric tree with a primary measure and a guardrail; results and the decision by the pre-set rule; tradeoffs with alternatives; and a retrospective with a pattern, a practice and a check. In the spoken version, the partner should be able to ask "how did you know that?" at each stage and get a number or a source. Mark anything you cannot support as an assumption.

Alternative solution. If you have no shipped project, use a labeled hypothetical for the full template and a real, smaller story for one stage.

Tradeoffs. The template takes preparation and survives probing.

13. Practice protocol

Timed drills (three a week for two weeks).

- *Week 1: stages and thresholds.* Use IQ-C30-01, IQ-C30-02, IQ-C30-03, EX-C30-01, EX-C30-02 and EX-C30-05. Fill in the journey template for a product idea, with a kill-criteria card written before you decide anything.

- *Week 2: economics, AI and the behavioral version.* Use IQ-C30-04 to IQ-C30-08 and EX-C30-03, EX-C30-04, EX-C30-06, EX-C30-07 and EX-C30-08. Compute the economics by hand, and rehearse the 30-minute idea-to-launch answer with a partner who probes.

With a partner. Ask your partner to say "How did you know that?" and "What would have made you stop?" at random, and to ask about one non-goal and one thing that went wrong.

Feedback checklist (yes, partly, no).

1. Did I give evidence for the problem, with a number?
2. Did I name the riskiest assumption and why?
3. Were thresholds and a middle zone written before the result?
4. Did the spec have non-goals and acceptance criteria?
5. Did I give a metric tree with a guardrail?
6. Did I decide by the pre-set rule and say where I used judgment?
7. Did I include non-feature requirements early?
8. Did my retrospective name a pattern, a practice and a check?

Answer journal. Record the date, the prompt, the stage where I was weakest, the number I could not produce, my score and the one fix.

14. Implementation playbook

Use the journey template on a real project over three weeks. If you have no current project, use a feature bet or a side project.

Days 1 to 3. Pick one product idea or one recently shipped feature. Fill in parts 1 and 2 of the template: context and users, and the discovery evidence with a named riskiest assumption.

Week 1. Write the kill-criteria card for the next test and the spec excerpt (problem statement, goals, non-goals, acceptance). Share both with an engineer and a data partner and ask what is missing.

Week 2. Run the test or review the result. Build the metric tree with a guardrail. Decide by the pre-set rule, and record the decision and the middle-zone judgment.

Week 3. Write the tradeoffs and the retrospective with a pattern, a practice and a check. Record it as a one-page journey and rehearse it aloud.

Deliverables that become stories: the completed template (a ready answer to "tell me about a product you shipped"), the kill-criteria card with its result (a time you decided by a rule written first) and the retrospective (a changed practice with a check). Keep every number real and credit partners.

15. Key takeaways and further reading

Key takeaways

1. An end-to-end story has a through-line: evidence for the problem, the riskiest assumption, a cheap test with thresholds set first, a spec with non-goals and acceptance criteria, a metric tree with guardrails, a decision by the pre-set rule, and a retrospective that changes a habit.
2. In all five journeys the riskiest assumption was not the build (willingness to pay and trust, the buying process, supply behavior, adoption, reliability and review), and a manual or narrow first version tested it cheaper than building.
3. Non-feature requirements (security review, consent, payment and credit rules, permissions, injection defenses) shape the plan and belong in it at the end of discovery.
4. Reliability multiplies down chains, reviewers must be tested with seeded errors, and autonomy is earned per band by supervised evidence with demotion on error.
5. Public phase models (discovery, alpha, beta, live, with assessments) formalize staged risk reduction; use the gates, write the exit criteria first and keep reviewing after launch. Use the journeys to learn the habits and tell your own story.

Further reading

- The GOV.UK Service Standard page; GDS blog posts of 2014, 2017 and 2020 on the standard and on assessments; a UK department's service-assessment page; and the New South Wales and Scottish government pages that restate the phases. Sources for CS-C30-02 (confirmed in Phase 5, October 2026).
- Eric Ries, *The Lean Startup* (2011), and Marty Cagan, *Inspired* (2017), on testing assumptions before building [V: Ries: Crown Business, 13 September 2011, ISBN 9780307887894; Cagan: 2nd edition, Wiley, December 2017].
- Chapters C05, C06, C16, C17, C24, C25, C26, C27 and C29 of this book, which teach the parts the journeys connect.